DA2026_PRJ2: Compiler Register Allocation

Graph-Coloring Heuristics

Beatriz Remondes Nuno Coimbra Simão Ribeiro

Faculty of Engineering, University of Porto

Spring 2026

Outline

Project Overview

- **Goal:** Implement a compiler back-end tool for global register allocation.
- **Approach:** Graph-coloring heuristics.
- **Pipeline:**
  1. Parse live ranges from 3-address intermediate representation.
  2. Build **live webs** (merging overlapping ranges).
  3. Construct an **interference graph**.
  4. Assign webs to N available registers.
- **Fallback Strategies:** Spilling, Splitting, and a Hybrid "Free" strategy when allocation fails.

The Graph Class: Conceptual Decisions

- **Base Structure:** Utilized the generic `Graph<T>` and `Vertex<T>` templates provided in practical classes.
- **Alterations & Additions:**
  - **Undirected Semantics:** Interference graphs are inherently undirected. Edge additions ensure symmetry ($A \to B$ and $B \to A$).
  - **DOT File Emission:** Added `emitDOTFile()` to export the graph to Graphviz format for visual debugging and presentation.
  - **Degree Tracking:** Integrated degree management (`getIndegree()`) crucial for the *Smallest-Degree-First* heuristic and the Simplify phase.
  - **Mutable Priority Queue:** Integrated `MutablePriorityQueue` to efficiently extract minimum degree nodes during the simplification stack building.

Spilling Heuristic (T2.2)

- **Objective:** Move uncolorable webs to memory to reduce graph chromatic number.
- **Spill Metric:** $\frac{\text{Degree(Node)}}{\text{Live\_Range\_length(Node)}}$
- **Rationale:**
  - High degree: removes maximum interference.
  - Short live range: minimizes memory load/store overhead.
- Iterates from 1 to K spilled webs, stopping when the graph becomes N-colorable.

Splitting Heuristic (T2.3)

- **Objective:** Divide a web into two smaller webs to reduce localized interference.
- **Maximum Interference Bottleneck:**
  - Analyzes internal execution points of the uncolorable web.
  - Finds the specific program line with the highest simultaneous interference.
- **Rationale:** Splitting at this exact point severs the most edges in the interference graph in a single operation.

"Free" Algorithm Strategy (T2.4)

- **Priority-Based Hybrid Approach.**
- Replaces standard stack with a *Smallest-Degree-First* priority queue.
- **Dynamic Impasse Resolution:**
  - **High Density (Near-Clique):** Triggers Spilling.
  - **Localized Clusters:** Triggers Splitting.
- Prevents aggressive splitting of highly connected graphs and unnecessary spilling of easily splittable webs.

Challenging Aspects of Implementation

- **Web Construction & Merging:**
  - Handling continuous vs. discontinuous live ranges accurately.
  - Merging variables with overlapping scopes efficiently $O(R^2 \cdot L)$.
- **Splitting Logic:**
  - Ensuring semantic correctness when dividing a web (part 1 & part 2).
  - Updating the interference graph dynamically without a full rebuild.
- **Hybrid Strategy Tuning:**
  - Balancing the heuristics to correctly identify when to Spill vs. Split.

Live Demonstration

Time for a Live Demo!

- **Interactive Mode:** We will run our allocator using the step-by-step interactive menu.
- **Test Cases:** Resolving a sample dataset of live ranges and available registers.

Conclusion

- Successfully implemented a robust register allocator using graph coloring.
- Extended the base `Graph` class for advanced heuristic support.
- The "Free" strategy optimally balances spills and splits.